

The `useIdleTimer` hook in `react-idle-timer` enables tracking user inactivity (idle/active states) in React applications. Using `ref` with `useIdleTimer` allows you to directly access the timer's methods (like `start`, `reset`, `pause`) or properties, often required for complex interaction scenarios, such as creating custom session timeout modals. [GitHub +4](#)

javascript

```
import { useRef } from 'react'
import { useIdleTimer } from 'react-idle-timer'

function App() {
  // 1. Create a reference for the idle timer
  const idleTimerRef = useRef(null)

  const onIdle = () => {
    console.log('User is idle')
  }

  // 2. Pass the ref in the configuration object
  useIdleTimer({
    ref: idleTimerRef,
    onIdle,
    timeout: 1000 * 60 * 5, // 5 minutes
    throttle: 500
  })

  return <div>My App</div>
}
```

Key Considerations for `useIdleTimer({ ref })`

- **Accessing Methods:** Once set up, you can call methods like `idleTimerRef.current.reset()` from anywhere in your component to manually reset the timer.
- **Component-Level Binding:** While `useIdleTimer` is a hook, `ref` allows it to function similarly to the older `IdleTimer` component API, allowing for direct DOM/instance manipulation.
- **Requirement:** It is essential to use `useRef` from React to create the reference object passed to the hook. [🔗](#)

Useful `useIdleTimer` Options [🔗](#)

- `timeout` : Time in milliseconds before the user is considered idle (default is 0).
- `onIdle` : Function fired when the timer goes idle.

- `onActive` : Function fired when the user returns.
- `events` : List of DOM events to listen for (e.g., `[ 'mousemove', 'keydown' ]`).
- `crossTab` : Syncs idle state across multiple browser tabs. [Stack Overflow +4](#)